Autonomous Table Robot Using RTAM-Map and Canny Edge Detection

Ann Maria Johnson
University of Birmingham
Email: axj447@student.bham.ac.uk

***Abstract*—This project focuses on developing a tabletop robot that is capable of autonomous navigation, avoiding obstacles and edges. Using the robust capabilities of ROS Noetic, this approach combines two techniques: SLAM RTAB-map framework for mapping and navigation, and Canny algorithm for edge detection. Designed for controlled indoor environments, the system lays an excellent foundation for future development with additional capabilities such as a robot that arranges objects on a table or even as an automated table-cleaning robot. Turtlebot3-waffle pi with RGB-D camera is utilized for RTAB mapping, along with computer vision methods.**

***Index Terms*—*Keywords:*- RTAM-Map, SLAM, Canny Algorithm**

I. Introduction

SLAM has seen multiple advancements over the years and has improved with multiple mapping techniques such as Gmapping, Hector SLAM, Cartographer, RTAB-Map so on. This paper aims to implement a tabletop robot that utilizes RTAB-map for autonomous navigation. The robot used is Turtlebot3 waffle pi as it has both a RGB-D camera and LiDAR capabilities. Since the robot is placed on top of a table, it is also expected to not fall off the table. To implement this depth data from the RGB-D camera can be utilized. Since the camera is mounted on top of the Turtlebot3 waffle pi, edge detection was the next best method to follow as RTAB-Map does not detect objects on the surface. By detecting the edges using the Canny algorithm the robot can navigate the table without falling off. RTAB- Map is chosen as it has 2D/3D mapping with advanced sensor fusion. In 3D mapping, it uses point cloud-based object detection which combines the RGB images, with depth data and the odometry pose to accurately map and navigate the environment. This is discussed in detail in the following sections. In this project, 2D maps are created using the laser data and effective mapping, navigation and loop closure is achieved. Along with 2D maps, edge detection is also done for autonomous navigation of the table robot. Analysis of two different worlds is done in this project. The first world is a symmetrical one with identical objects, and the second one is an asymmetrical world with some different objects. No object below the camera level is used since RTAB-Map does not detect that object. The algorithm is lightweight and does not take much processing power as it uses 2D mapping and real-time edge detection.

II. Related Work

The novel approach proposed as SLAM++ by Salas-Moreno et al. (2013) [1] discusses how the object-oriented 3D SLAM paradigm is efficient for indoor applications with repeated objects. This method not only effectively maps and navigates in a closed, dense environment but also reduces computational overload by eliminating complex map overlapping and optimisations. Semantic segmentation is a huge advantage in detecting partially hidden objects and this can be leveraged in an environment like a tabletop to effectively detect moved objects. This technology can also be used for robot fall prevention, by effectively segmenting the surroundings of the table. This system produces an object-centred map which significantly increases environmental understanding, precision in localization and object interaction. These methods can be incorporated into the future work of tabletop robots.

A comparative study on two vSLAM methods: ORB-SLAM2 and RTAM-Map by Ragot et al. concludes that RTAB mapping performs well in long-term mapping and loop closure detection, whereas ORB-SLAM2 demonstrates high precision in structured and static settings [2]. Thus, ORB-SLAM2 is also an excellent choice for a tabletop robot.

III. Algorithms and Frameworks

A. Navigation using RTAB

The autonomous navigation algorithm given in the $goalpose3.py$ file in the scripts folder of the turtle bot package sets three goal points. The robot simultaneously maps and navigates the environment based on the map saved previously. The actionlib framework is used to send commands to the movebase package. The algorithm uses two types of movebase messages, the first one is MoveBaseAction and the second is MoveBaseGoal. A node is initialized, with a client waiting for the MoveBaseAction message from the server. The goal for the robot to navigate to is set through the MoveBaseGoal message with the x, y, z position and orientation data being sent. Callbacks implemented for any server interruptions.

B. Edge Detection using Canny

The obstacle avoidance system leverages the Canny algorithm to effectively detect the edges of the table and objects on it. OpenCV converts the image messages to cv2 data which are then converted to grey-scale images, the Canny algorithm is implemented on these greyscale images. The shape of the

frame in which this grey image is displayed is divided into twenty parts based on height to evaluate the lower pixels of the frame. The pixel values are counted using NumPy and a conditional statement is run on these to effectively make decisions for the movement of the robot. The conditional statement is divided into five situations. The first one detects the objects on the table when it is too close making the robot stop, move back and turn. The next one is a go condition when the objects are not too close. The next two conditions detect the edges of the table and tell the robot when to move and when to stop, move back and turn. The last condition is when the robot is too close to the edge, this completely stops the robot moves it back and makes it turn almost 180 degrees. The publisher node publishes the velocity for the corresponding movement.

C. Evaluation of RTAB-Map SLAM Framework

Real-Time Appearance Based Mapping (RTAB-Map) supports both visual and lidar SLAM, providing 3D and 2D information suitable for indoor and outdoor applications. It is an open source library with good support and user-friendly implementation resources. RTAB-MAP was first proposed as an appearance-based loop closure detection approach and later spanned to implement SLAM [3]. The map created using RTAB-Map consists of nodes which is a combination of RGB images along with depth information and corresponding odometry pose. Odometry pose refers to the position of the robot x, y, z coordinates) and orientation (pose) in the environment. The links in RTAB-Map represent transformations between nodes in the graph. Loop closure is achieved by comparing the latest image with all previously captured images, while updating the graph. A point cloud is generated from the RGB images and depth information, which is associated with each node that is then transformed based on the pose of that node while updating the map. The final 3D map is thus generated based on these transformed point clouds.

There are several key components in RTAB-Map:

- Visual Features: These help in matching and detecting images or frames across time.
- 3D Reconstruction: 3D maps are generated from the depth data collected from RGB-D cameras
- Loop Closure Detection: The drift in the map is corrected by recognizing previously visited areas in the environment.
- Pose Graph Optimization: The map consistency is improved by correcting the trajectory of the robot.

D. Mathematical Formulation:

- Localization and Pose Estimation:
  The visual information from the RGB-D camera is used to localize the robot. The Visual Odometry (VO) calculations are done given the sequence of images and depth data to estimate the robot's motion between two consecutive frames. Let the robot's pose at time t be represented by $T_t = [R_t, t_t]$, where R_t is the rotation matrix and t_t is the translation vector. The motion between two frames t and $t-1$ can be expressed as:

$$T_t = T_t - 1 \cdot \Delta T_t - 1, t \tag{1}$$

  where $\Delta T_t - 1, t$ is the relative motion computed by visual odometry.

- Loop Closure:
  This technology helps correct drift in maps. The current frame is compared to the previously stored frame when the robot revisits a location in the map. When the similarity score exceeds a threshold, RTAB-Map detects a loop closure. The pose graph is then updated to incorporate the new loop closure, correcting the robot's trajectory. Let I_1 and I_2 be two frames based on visual descriptor $f(I)$ of the images. The similarity Measure is given as:

$$S(I_1, I_2) = sim(f(I_1), f(I_2)) \tag{2}$$

  where sim could be based on the Euclidean distance between feature vectors or other distance metrics.

- Pose Graph Optimization:
  The trajectory and map of the robot are improved in this method. Reducing the errors from the overtime drift of the robot's pose is the aim of this optimization.
  - **Pose Graph Representation:** The graph represents the pose and constraints of the robot obtained from odometry and loop closures. Each node represents a pose, and each edge represents a constraint between poses (either from odometry or from loop closure detection).

$$G = T_0, T_1, ...T_n \tag{3}$$

    where each pose T_i is connected by constraints.
  - **Graph Optimization:** The optimization problem is formulated as minimizing the total cost function, which incorporates the difference between consecutive poses and loop closures:

$$\min_{T_0, T_1, ...T_n} \sum_{i=1}^{n} \|T_i - T_{i-1} \cdot \Delta T_i\|^2 + \sum_{\text{loop closures}} \|T_i - T_j\|^2 \tag{4}$$

    where each pose ΔT_i is the estimated transformation between poses from visual odometry, and the second term minimizes the difference between loop closure poses.

- 3D Mapping:
  The depth information from the RGB-D camera is used to create the 3D map. Given the depth map $D(x, y)$ and the corresponding RGB image $I(x, y)$, the 3D coordinates of each pixel can be derived using the pinhole camera model. For a pixel at locationx, y with depth $d = D(x, y)$, the 3D coordinates are computed as:

$$X = \frac{(x - c_x) \cdot d}{f_x}, Y = \frac{(y - c_y) \cdot d)}{f_y}, Z = d \tag{5}$$

  where (c_x, c_y) is the camera's principal point, and f_x, f_y are the focal lengths of the camera.

E. Evaluation of Canny Edge Detection

Canny Edge Detection algorithm has the ability to detect a wide range of edges in images while minimizing noise see [5]. The first step involves using a Gaussian filter to reduce the noise in the image. The filter blurs the image, smoothing out rapid intensity changes and removing high-frequency noise, which would otherwise interfere with edge detection. Next the gradient of the image intensity is calculated at each pixel using the Sobel operator. Following this the algorithm goes through each pixel in the gradient image and checks if the gradient magnitude at that pixel is greater than the magnitudes of the two neighboring pixels in the gradient direction. If not, it is set to zero. This is to thin the edges. Genuine edges and noise are separated by setting threshold values and classifying the pixels. Final step is to connect the weak edges to strong edges. If a weak edge is connected to a strong edge, it is considered part of the edge, otherwise, it is discarded. This helps to remove isolated weak edges that do not contribute to the overall structure of the object. This framework combined with RTAB-map provides great advantage for navigating on elevated environments.

IV. Experimental Results

The Turtlebot3-waffle-pi with RGB-D camera was deployed in two different Gazebo worlds. This was to test if loop closure would be better implemented in identical or nonidentical environments. The first world contains identical beer cans, symmetrically placed on the table. The robot can successfully map and navigate in this environment but localization becomes strenuous due to the identical symmetry. This also makes the loop closure quite taxing. The second environment contained unsymmetrical and nonidentical objects. The robot maps and navigates this environment successfully. Localization is also carried out with ease. Loop closure is still not completely accomplished in this environment as well.

Edge Detection is carried out separately from RTAB mapping. This is because the $/camera/rgb/imageraw$ topic couldn't be accessed simultaneously by both applications. This can be fixed in future implementations. The edge detection works well when the robot is orthogonal to the edge of the table. But for the corners, the robot fails to detect the edges. This is due to the difference in number of bright pixels at corners compared to the straight edges. This can be fixed by calibrating the algorithm proposed. Extra lighting is provided for edge detection, while this is not needed for RTAM-mapping.

The RTAB-Mapping does not detect the bowl placed on the table. This might be because the sensor is being placed on top of the turtlebot. The bowls are detected in edge detection by the Canny algorithm.

Thus by combining both these frameworks the robot can successfully navigate the table autonomously. The current program does not do this as it was unable to run both programs simultaneously. Other vSLAM frameworks as as ORB-SLAM could prove to be more efficient as they can incorporate more computer vision capabilities.

V. Conclusion

In conclusion, when combined, RTAB-mapping and Canny algorithm can successfully be deployed for a table robot. ORB-SLAM is also an alternative vSLAM method that can be adopted for this environment. The robot's size and the camera's placement play a crucial role in table robot navigation. In this project, a default robot is used which limits the movement and camera capabilities to retrieve the environment data. It is also seen after evaluation of the different worlds that loop closures would be more successful in unsymmetrical environments with nonidentical objects. Thus this would correspond to real-world environments with several objects. Furthermore, object detection algorithms such as YOLO can be implemented on top of this. This could help in an application where the robot needs to arrange the table or collect garbage on the table.

References

[1] R. F. Salas-Moreno, R. A. Newcombe, H. Strasdat, P. H. J. Kelly, and A. J. Davison, *SLAM++: Simultaneous Localisation and Mapping at the Level of Objects*, in *Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, Tokyo, Japan, 2013, pp. 1358–1364, doi: 10.1109/IROS.2013.6696747.

[2] N. Ragot, R. Khemmar, A. Pokala, R. Rossi, and J. Y. Ertaud, "Benchmark of Visual SLAM Algorithms: ORB-SLAM2 vs RTAB-Map," in *Proceedings of the International Conference on Intelligent Robots and Systems (IROS)*, 2023, Saint Etienne du Rouvray, France.

[3] M. Labbé and F. Michaud, "RTAB-Map as an Open-Source Lidar and Visual Simultaneous Localization and Mapping Library for Large-Scale and Long-Term Online Operation," *Journal of Field Robotics*, vol. 36, no. 2, pp. 416–446, 2019, doi: 10.1002/rob.21831.

[4] C. Cadena et al., "Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age," *IEEE Transactions on Robotics*, vol. 32, no. 6, pp. 1309–1332, 2016, doi: 10.1109/TRO.2016.2624754.

[5] X. Zhou, Y. Zhang, and W. Chen, "A Parallel Canny Edge Detection Algorithm Based on OpenCL Acceleration," *PLOS ONE*, vol. 11, no. 4, pp. 1–16, Apr. 2016, doi: 10.1371/journal.pone.0153655.